APM 2025-2026 : Contrôle Continu 1 - Sujet

18 octobre 2026

Table des matières

I Questions de cours **2**
Q.1 . . . 2
Q.2 . . . 2
Q.3 . . . 2
Q.4 . . . 2
Q.5 . . . 2

II Succession de kernels **2**
Q.6 . . . 3

III Safe/Unsafe **3**
Q.7 . . . 3
Q.8 . . . 3
Q.9 . . . 4
Q.10 . . . 4

IV Réduction somme en CUDA **5**
Q.11 . . . 5

Durée indiquée sur le sujet : 3 heures. Enseignant : Julien Jaeger. Toutes les réponses doivent être justifiées ; la qualité de la rédaction est prise en compte. Aucun barème n'est visible sur les quatre photos.

Les identifiants, dimensions, décalages de pointeurs et appels CUDA du sujet sont conservés, même lorsqu'ils semblent incorrects. Réponses : corrigé.

I Questions de cours

Q.1

Que fait la fonction cudaMallocHost ? Quelles sont les différences avec la fonction cudaMalloc ? et la fonction malloc ?

Q.2

Que signifie faire un accès coalescé à la mémoire ?

Q.3

Exprimer le calcul de l'indice global d'un thread CUDA en considérant que les blocs sont définis en 3D avec des tailles de BLOCK_X, BLOCK_Y et BLOCK_Z, et que la grille est définie en 2D avec des tailles de GRID_X et GRID_Y.

Q.4

Lequel de ces mots clés n'est pas un attribut de données dans l'API CUDA.

1. `__device__`
2. `__shared__`
3. `__local__`
4. `__constant__`

Q.5

Quel est l'équivalent en OpenAcc du pragma OpenMP `#pragma omp target data map(to :a[0 :8])` ?

II Succession de kernels

Nous considérons onze kernels, s'exécutant et se succédant sur trois streams différents du même GPU, selon le graphe en Figure 1. Les flèches entre les kernels représentent les dépendances de données (par exemple, A produit des données utilisées par C et D).

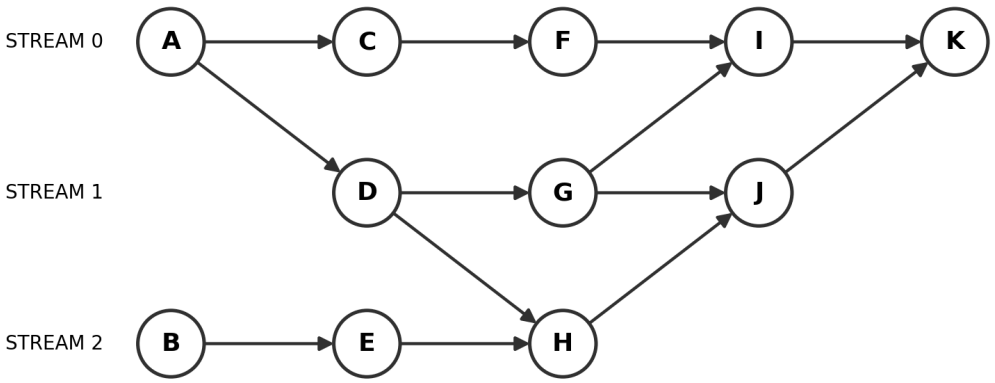


Figure 1. Graphe redessiné à partir de la page 2.

Q.6

Donner le code des appels de kernels avec les synchronisations minimales nécessaires pour avoir un comportement sûr du programme, et obtenir le bon résultat.

III Safe/Unsafe

Dites si les sections de codes CUDA suivantes risquent de provoquer des erreurs ou des comportements non-déterministes lors de leurs exécutions.

Q.7

code :

```
cudaStream_t stream[1];
cudaStreamCreate(&stream[0]);
int *htab, *dtab, *tmp;
int size = N*sizeof(int);
cudaMalloc((void**)&dtab, 2*size);
cudaMallocHost((void**)&tmp, size);
cudaMallocHost((void**)&htab, size);
memset(htab, 0, size);
cudaMemcpyAsync(dtab, htab, size, cudaMemcpyHostToDevice, stream[0]);
kernel<<<grid, block, 0, stream[0]>>>(dtab, tmp, N);
cudaMemcpyAsync(htab, dtab, size, cudaMemcpyDeviceToHost, stream[0]);
cudaStreamSynchronize(stream[0]); for(i=0; i<N; i++)
    printf("%d", htab[i]);
```

Q.8

code :

```
cudaStream_t stream[3];
cudaStreamCreate(&stream[0]); cudaStreamCreate(&stream[1]);
cudaStreamCreate(&stream[2]);
int *htab, *dtab;
int size = N*sizeof(int);
cudaMalloc((void**)&dtab, 6*size);
cudaMallocHost((void**)&htab, 7*size);
memset(htab, 0, 7*size);
cudaMemcpyAsync(dtab, htab, size, cudaMemcpyHostToDevice, stream[0]);
cudaMemcpyAsync(dtab + size, htab + size, 2*size,
                cudaMemcpyHostToDevice, stream[1]);
cudaMemcpyAsync(dtab + 3*size, htab + 3*size, 3*size,
                cudaMemcpyHostToDevice, stream[2]);
kernel<<<grid0, block0, 0, stream[0]>>>(dtab, N);
kernel<<<grid1, block1, 0, stream[1]>>>(dtab + size, 2*N);
kernel<<<grid2, block2, 0, stream[2]>>>(dtab + 3*size, 3*N);
cudaMemcpyAsync(htab, dtab, size, cudaMemcpyDeviceToHost, stream[0]);
cudaStreamSynchronize(stream[0]);
cudaMemcpyAsync(htab + size, dtab + size, 2*size,
                cudaMemcpyDeviceToHost, stream[1]);
cudaStreamSynchronize(stream[1]);
cudaMemcpyAsync(htab + 3*size, dtab + 3*size, 3*size,
                cudaMemcpyDeviceToHost, stream[2]);
cudaStreamSynchronize(stream[2]); for(i=0; i<7*N; i++)
    printf("%d", htab[i]);
```

Q.9

code :

```
int *htab, *dtab;
int size = N*sizeof(int);
cudaMalloc((void**)&dtab, 2*size);
cudaMallocHost((void**)&htab, size);
cudaMemcpyAsync(dtab, htab, size, cudaMemcpyHostToDevice);
kernel<<<grid, block>>>(dtab, N);
cudaMemcpyAsync(htab, dtab, size, cudaMemcpyDeviceToHost);
for(i=0; i<N-1; i++)
    htab[i] = htab[i+1];
htab[N-1] = 0;
```

Q.10

code :

```
cudaStream_t stream[2];
cudaStreamCreate(&stream[0]); cudaStreamCreate(&stream[1]);
int *htab, *dtab;
int sz_in_bytes = N*sizeof(int);
cudaMalloc((void**)&dtab, 2*sz_in_bytes);
cudaMallocHost((void**)&htab, 2*sz_in_bytes);
memset(htab, 0, 2*sz_in_bytes);
cudaMemcpyAsync(dtab, htab, sz_in_bytes,
                cudaMemcpyHostToDevice, stream[0]);
cudaMemcpyAsync(dtab + sz_in_bytes, htab + sz_in_bytes, sz_in_bytes,
                cudaMemcpyHostToDevice, stream[1]);
kernel<<<grid, block, 0, stream[0]>>>(dtab + sz_in_bytes, N);
kernel<<<grid, block, 0, stream[1]>>>(dtab, N);
cudaMemcpyAsync(htab, dtab, sz_in_bytes,
                cudaMemcpyDeviceToHost, stream[0]);
cudaStreamSynchronize(stream[0]);
cudaMemcpyAsync(htab + sz_in_bytes, dtab + sz_in_bytes, sz_in_bytes,
                cudaMemcpyDeviceToHost, stream[1]);
cudaStreamSynchronize(stream[1]);
for(i=0; i<2*N; i++)
    printf("%d", htab[i]);
```

IV Réduction somme en CUDA

Une réduction somme consiste à additionner toutes les valeurs d'un tableau. Une écriture séquentielle pourrait être la suivante :

```
float sum = 0;
for (int i = 0; i < ntot; i++)
    sum += tab[i];
```

On désire une implémentation CUDA, avec `tab` déjà sur le GPU : une réduction dans chaque bloc produit un tableau de sommes partielles, puis une seconde réduction de ce tableau produit la somme totale.

Q.11

Implémenter le kernel `reduce_kernel(float *in, float *out)` permettant de faire les sommes partielles par bloc. `in` est le tableau de valeurs à réduire dimensionné au nombre total de threads dans la grille, et `out` le tableau de valeurs réduites par bloc, dimensionné au nombre de bloc.

Donner les deux invocations successives de cette fonction permettant de faire d'abord la somme partielle par bloc, puis la somme totale à partir des sommes partielles.

Pour simplifier, nous considérons que le nombre de blocs et de threads par blocs sont des puissances de 2, et que la taille du tableau initiale est égale au nombre de threads.